# SmartBugs - Main Modification Report

Luciano Bercini

March 2026

## Contents

# 1 Context of the Project

The project focuses on the evolution of *SmartBugs*, an extensible framework designed for the automated analysis of Ethereum smart contracts. SmartBugs provides a unified interface to execute multiple vulnerability detection tools, such as static and dynamic analyzers, by encapsulating them within Docker containers.

The framework allows users to:

- Execute multiple analysis tools on one or more smart contracts
- Standardize the output of heterogeneous tools
- Automatically manage Solidity compiler versions

Currently, SmartBugs supports two main execution modes:

- Execution of all available tools
- Execution of a user-specified subset of tools

Despite its flexibility, SmartBugs lacks an intelligent mechanism to select the most appropriate tools based on the characteristics of the input smart contract. This often leads to inefficient analyses, unnecessary computational overhead, and redundant tool executions.

To address this limitation, this project proposes an extension of SmartBugs by integrating:

- A feature extraction tool (*SCsVulLyzer*)
- A set of pre-trained Machine Learning models

The goal is to enable an *intelligent tool selection mechanism* that automatically determines which analysis tools should be executed for a given smart contract.

# 2 Goals of the Project

The main goal of this project is to extend SmartBugs by introducing a new execution mode capable of automatically selecting the most suitable vulnerability detection tools based on the structural characteristics of a smart contract.

More specifically, the project aims to:

- Integrate *SCsVulLyzer* to extract features from Solidity smart contracts
- Load and execute a set of pre-trained Machine Learning models (stored as `.joblib` files)
- Use these models to predict the most appropriate analysis tools for different vulnerability categories
- Aggregate the predictions and automatically construct the list of tools to execute
- Invoke SmartBugs using the selected tools without modifying its core execution pipeline

The proposed solution introduces a new **intelligent command** that follows this workflow:

1. Extract features from the input smart contract
2. Select the relevant subset of features required by the models
3. Execute multiple ML models (one per vulnerability category)
4. Collect predicted tools
5. Filter results (remove duplicates and irrelevant outputs)
6. Execute SmartBugs with the selected tools

A key design goal is to ensure that the modification:

- Is modular and non-intrusive
- Preserves backward compatibility
- Minimizes changes to the existing execution pipeline

# 3 Analysis through Reverse Engineering

This section presents the analysis of the SmartBugs framework through reverse engineering. The goal is to understand the architecture, identify key components, and evaluate how the proposed modification can be integrated.

## 3.1 Architectural Overview

SmartBugs follows a modular architecture composed of several Python modules, each responsible for a specific aspect of the system:

- **sb.cli**: handles command-line arguments and initializes execution
- **sb.settings**: manages configuration parameters
- **sb.smartbugs**: orchestrates the overall workflow
- **sb.tools**: loads and represents analysis tools
- **sb.tasks**: defines execution units (tasks)
- **sb.analysis**: coordinates execution of tasks
- **sb.docker**: executes tools inside Docker containers
- **sb.solidity**: handles Solidity compilation and pragma resolution

## 3.2 Package Diagram

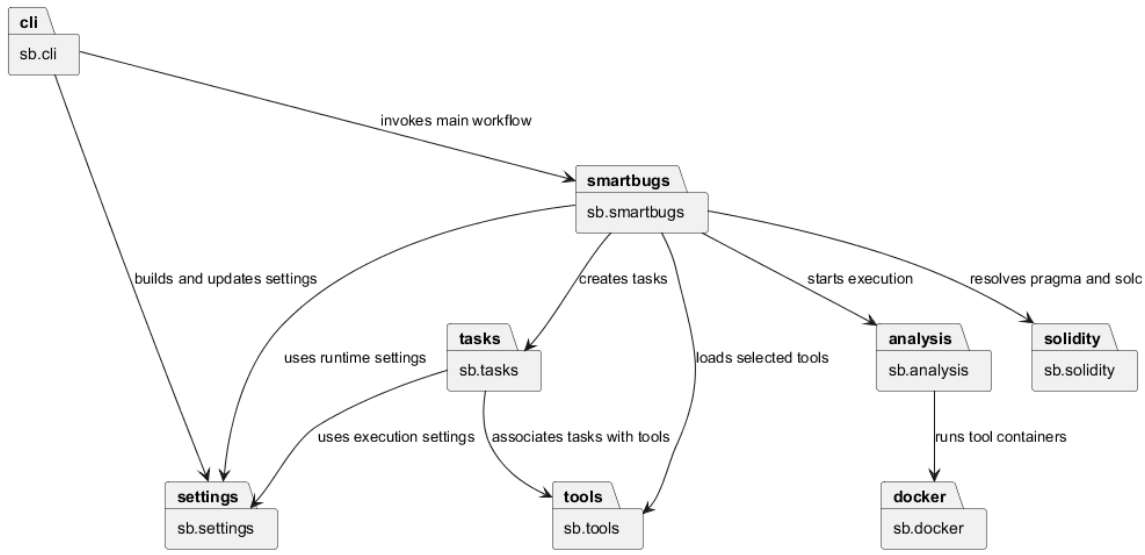


Figure 1: SmartBugs Package Diagram

The package diagram shows the main SmartBugs modules and their dependencies. sb.cli is the entry point of the system and is responsible for parsing command-line arguments and initializing the execution settings. sb.smartbugs acts as the main orchestrator, coordinating tool loading, pragma and solc resolution, task creation, and analysis execution. The execution phase is delegated to sb.analysis, which relies on sb.docker to run the selected tools inside containers. Auxiliary modules such as sb.io, sb.logging, and sb.errors have been omitted for readability, since they provide support services rather than core logic.

This architecture clearly separates:

- Configuration management
- Task generation
- Execution logic

This separation is crucial for enabling future extensions.

### 3.3 Dependency Analysis

To further analyze the relationships identified in the package diagram, we constructed a dependency matrix that quantifies the interactions between the core modules. Each dependency count is derived from explicit source-level interactions, including module imports, type-related imports, and concrete uses of imported modules or classes. String-based type annotations were not counted.

The dependency matrix was derived by analyzing the source code of the main modules and counting the dependencies between them. Each cell $(i, j)$ represents the number of times module $i$ depends on module $j$. In this study, the analysis focuses only on the core modules involved in the orchestration and execution workflow, while auxiliary modules such as `sb.io`, `sb.logging`, `sb.errors`, and `sb.cfg` were omitted for readability.

Table 1: Dependency Matrix of the Core SmartBugs Modules

| **Module** | **cli** | **settings** | **smartbugs** | **tools** | **tasks** | **analysis** | **docker** | **solidity** |
|---|---|---|---|---|---|---|---|---|
| cli | 0 | 3 | 2 | 0 | 0 | 0 | 0 | 0 |
| settings | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| smartbugs | 0 | 3 | 0 | 3 | 3 | 2 | 3 | 5 |
| tools | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| tasks | 0 | 2 | 0 | 2 | 0 | 0 | 0 | 0 |
| analysis | 0 | 2 | 0 | 0 | 4 | 0 | 2 | 0 |
| docker | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| solidity | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

The dependency matrix highlights the structural relationships between the core modules of SmartBugs. Each cell represents the number of dependencies from a module (row) to another module (column).

The analysis clearly shows that `sb.smartbugs` is the central orchestrator of the system, as it exhibits dependencies toward almost all other core modules, including `settings`, `tools`, `tasks`, `analysis`, `docker`, and `solidity`. This confirms its role as the main coordination component responsible for managing the overall workflow.

The `sb.cli` module primarily depends on `settings` and `smartbugs`, reflecting its responsibility for parsing user input and delegating execution to the core system.

The `sb.analysis` module shows dependencies on `tasks` and `docker`, which is consistent with its role in executing analysis tasks through containerized tools.

Modules such as `settings`, `tools`, and `solidity` exhibit no outgoing dependencies within this subset, indicating that they act as supporting or utility components.

In particular, the strong dependency between `sb.smartbugs` and `sb.solidity` highlights the importance of Solidity-specific preprocessing in the analysis pipeline.

Overall, the matrix confirms a modular architecture with a centralized orchestration layer. This structure is particularly suitable for extension, as new functionalities, such as the integration of machine learning-based tool selection, can be introduced at the orchestration level without requiring significant modifications to the execution layer.

### 3.4 Class Diagram

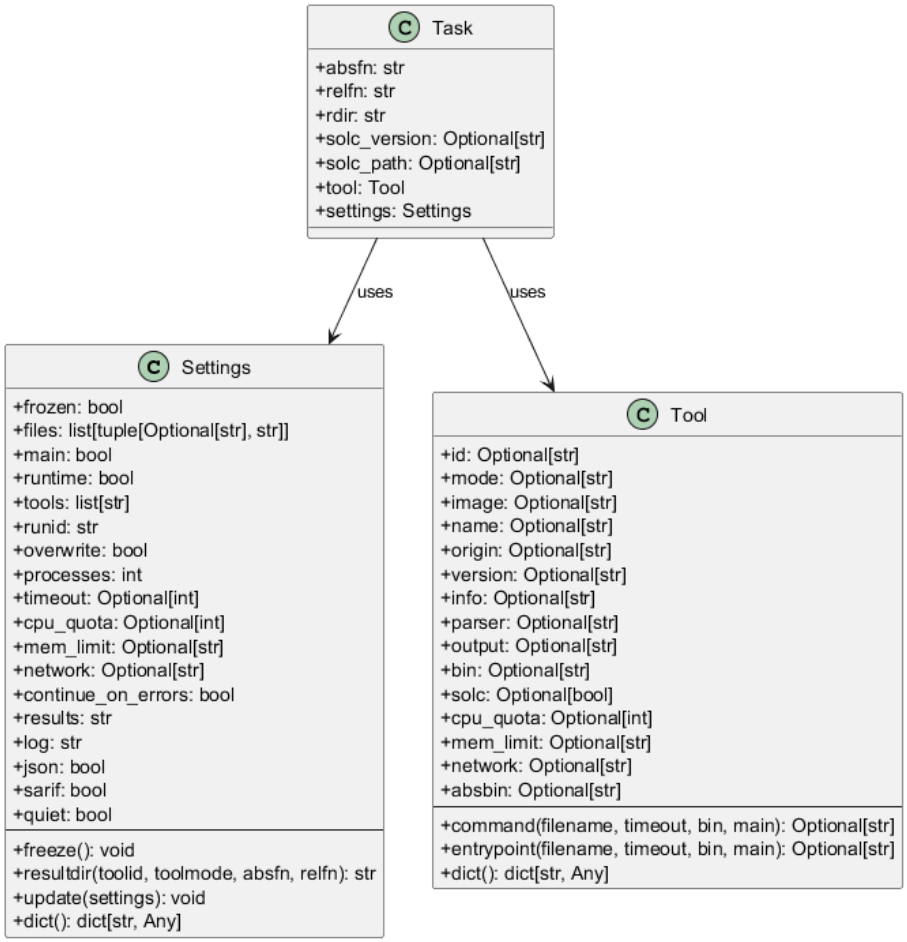


Figure 2: Core Class Diagram of SmartBugs

The class diagram focuses on the main data abstractions of SmartBugs that are directly involved in the configuration and execution of the analysis workflow.

**Settings** represents the global configuration of a SmartBugs execution. It stores all parameters required to control the analysis, including the list of input files, selected tools, execution constraints (e.g., timeout, CPU and memory limits), and output configuration. The class provides methods to update the configuration from external sources, normalize and freeze runtime parameters, and generate concrete result directory paths. This makes *Settings* the central coordination object shared across all phases of execution.

**Tool** models a single analysis tool supported by SmartBugs. It encapsulates both descriptive metadata (such as identifier, version, Docker image, and execution mode) and behavioral aspects, namely how the tool is invoked. The invocation logic is implemented through templated command and entrypoint definitions, which are instantiated at runtime depending on the specific task being executed. This abstraction allows SmartBugs to uniformly manage heterogeneous tools while keeping their execution configurable and extensible.

**Task** represents the fundamental unit of execution in the system. Each task corresponds to the application of a specific tool to a given input file. It aggregates all the information required for execution, including file paths, output directory, optional Solidity compiler configuration, the associated tool, and the global settings. In this way, *Task* acts as a bridge between configuration and execution, enabling the analysis engine to process multiple tool-file combinations in a structured manner.

The relationships between these classes reflect a clear separation of concerns. A *Task* depends on both *Tool* and *Settings*, combining them into an executable unit, while *Tool* and *Settings* remain independent abstractions. This design improves modularity and facilitates future extensions, such as introducing new tool selection strategies or modifying execution parameters, without altering the core execution model.

## 3.5 Sequence Diagram of the Current Workflow

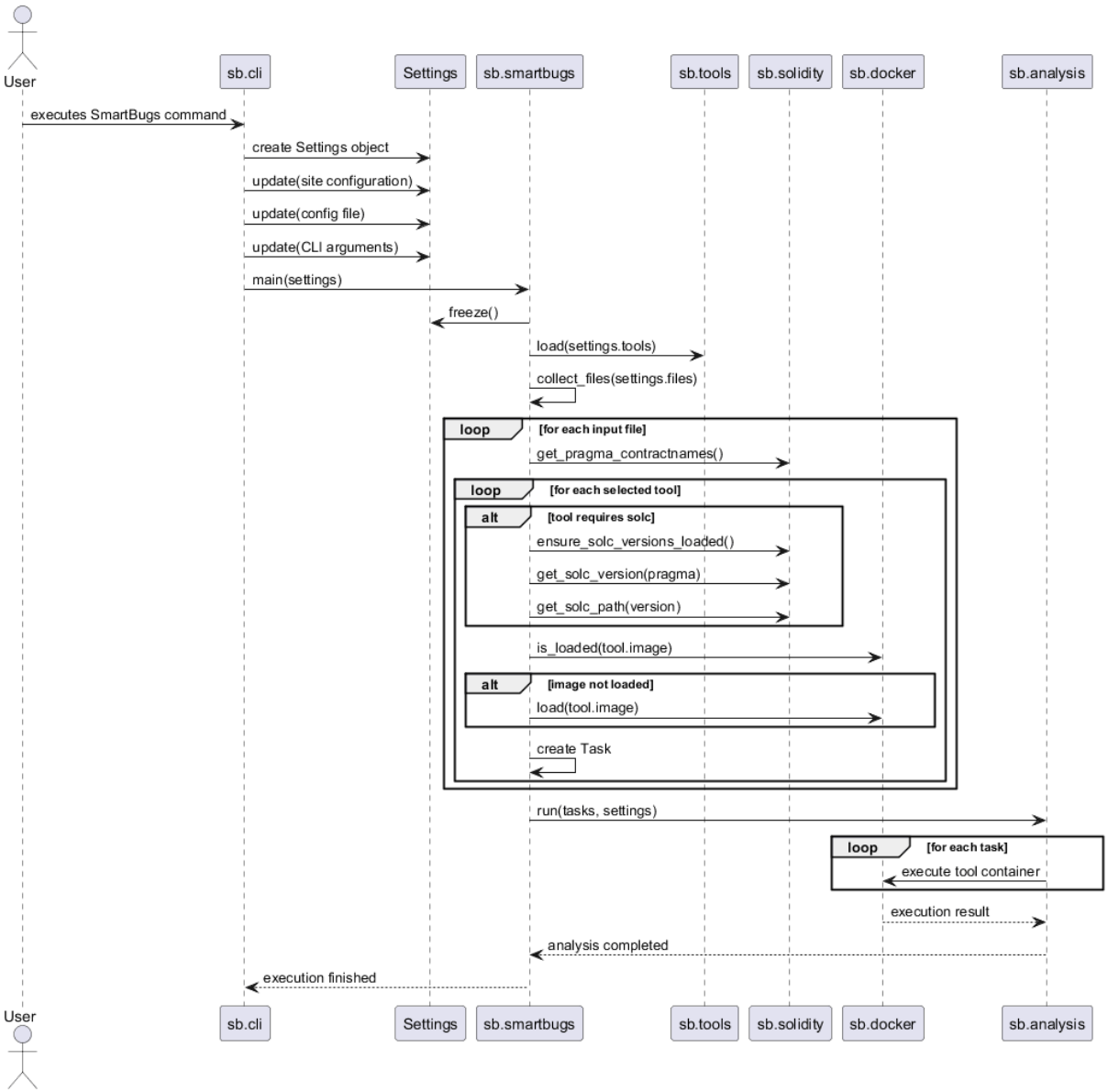


Figure 3: Current SmartBugs Execution Workflow

The sequence diagram illustrates the current execution workflow of SmartBugs. The process starts when the user invokes the framework through the command line interface. The `sb.cli` module creates a `Settings` object and progressively updates it using site-level configuration, optional configuration files, and command-line arguments.

Once the configuration is fully prepared, control is transferred to `sb.smartbugs`, which freezes the runtime settings, loads the selected tools, and collects the input files to be analyzed. For each input file, SmartBugs extracts Solidity-specific information, such as the pragma and contract names. Then, for each selected tool, it resolves the appropriate Solidity compiler when required, checks whether the corresponding Docker image is already available, and loads it if necessary. Finally, it creates the corresponding execution tasks.

After task generation, the execution phase begins. The list of tasks is passed to `sb.analysis`, which coordinates the execution of each task by invoking the appropriate Docker container through `sb.docker`. The results are then collected, and the overall execution terminates.

The diagram highlights a clear separation between configuration, task composition, and execution. This separation is important from an evolution perspective, since it suggests that modifications to tool selection can be introduced before the execution phase without necessarily affecting the existing task execution pipeline.

## 3.6 Analysis of External Components

Two external components were analyzed:

### 3.6.1 SCsVulLyzer

SCsVulLyzer is a feature extraction tool capable of analyzing Solidity smart contracts and producing a large set of features, including:

- AST-based features
- Opcode-based features
- Bytecode characteristics

Experimental validation confirmed that SCsVulLyzer can be executed correctly and produces consistent outputs.

### 3.6.2 Machine Learning Models

Eight pre-trained models were provided, each corresponding to a specific vulnerability category (SWC).

Through reverse engineering, the following observations were made:

- Models are implemented as **scikit-learn pipelines**
- Each model requires **41 input features**
- These features correspond to a subset of SCsVulLyzer outputs
- The output of each model is a **tool name** (e.g., `slither`, `mythril`, etc.)

Additionally:

- Models require **scikit-learn version 1.6.1** for compatibility
- Each model includes preprocessing steps (e.g., scaling, encoding)

## 3.7 Sequence Diagram of the Proposed Workflow

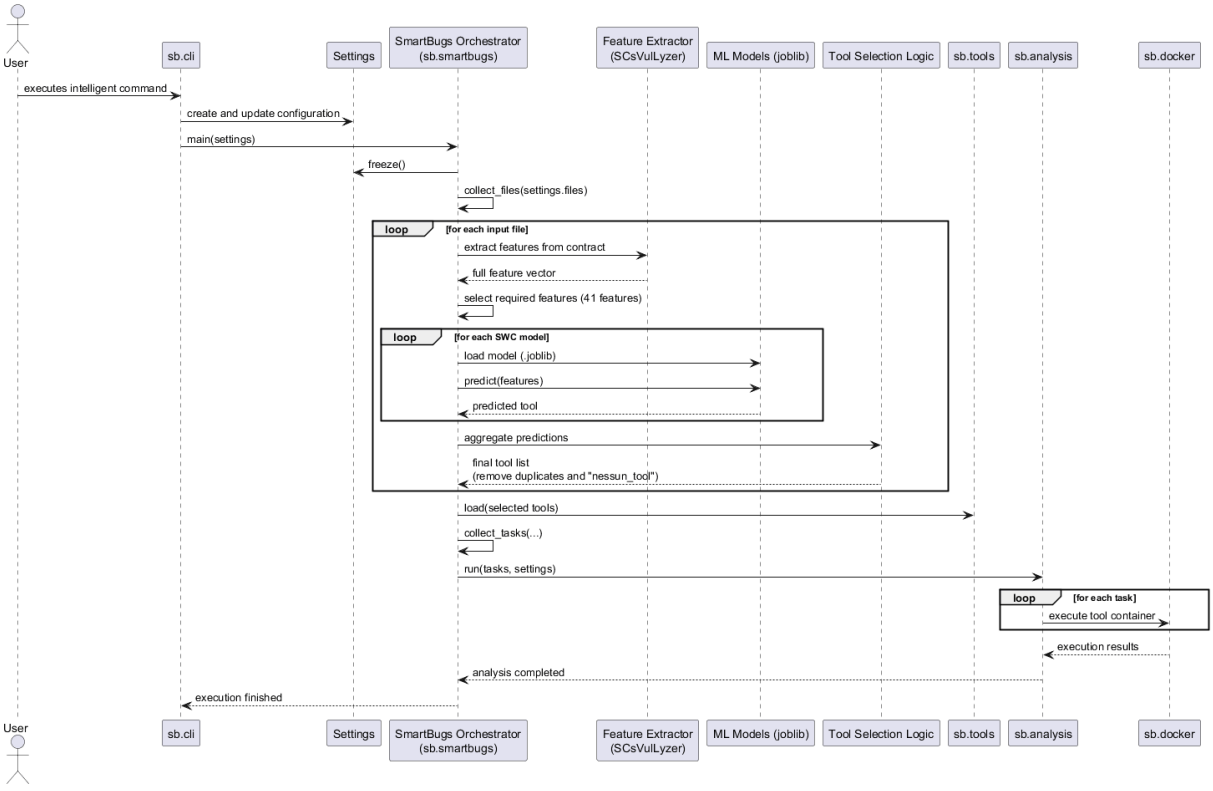


Figure 4: Proposed Intelligent SmartBugs Workflow

The proposed workflow extends the original SmartBugs execution pipeline by introducing an intelligent tool selection phase based on machine learning models. The process still starts from the command-line interface, where the configuration is created and finalized as in the original system.

After collecting the input files, the system introduces a new feature extraction step. For each smart contract, features are extracted using SCsVulLyzer. These features describe structural and behavioral aspects of the contract, including AST properties and opcode statistics.

From the extracted feature vector, only the subset of features required by the trained models is selected. This subset corresponds to the input expected by the machine learning pipelines.

The system then executes multiple pre-trained models, each associated with a specific vulnerability category (SWC). Each model receives the same feature vector and predicts the most suitable analysis tool for that category. This results in multiple tool predictions for a single contract.

The predicted tools are then aggregated and filtered. Duplicate tools are removed, and non-informative predictions (e.g., "nessun_tool") are discarded. The resulting set represents the tools that are expected to be most effective for analyzing the given contract.

Once the tool selection phase is completed, the workflow returns to the standard SmartBugs pipeline. The selected tools are loaded, tasks are generated, and the analysis is executed using Docker containers as in the original system.

This design preserves backward compatibility and maintains a clear separation between tool selection and execution. As a result, the existing execution infrastructure of SmartBugs remains unchanged, while the decision process for selecting tools becomes adaptive and data-driven.

### 3.8 Summary of Findings

The reverse engineering phase highlights that:

- SmartBugs is highly modular and suitable for extension
- The current limitation lies in manual tool selection
- External tools and models are compatible with the framework
- The integration can be achieved with limited architectural impact

## 4 Proposed Changes & Contributions

This section describes the proposed modifications introduced to extend the SmartBugs framework.

The goal of the project is to introduce an intelligent mechanism for automatic tool selection based on features extracted from smart contracts and predictions generated by machine learning models.

To improve modularity, maintainability, and impact analysis, the proposed solution is decomposed into three change requests (CRs), each addressing a specific aspect of the system.

### 4.1 CR_1: Integration of Feature Extraction Module

| | |
|---|---|
| **Description** | Integration of a feature extraction module (SCsVulLyzer) to automatically extract relevant characteristics from smart contracts. |
| **Motivation** | SmartBugs currently operates without any feature-based analysis. Introducing feature extraction is necessary to enable intelligent decision-making for tool selection. |
| **Priority** | High [X] Medium [ ] Low [ ] |
| **Effort** | High [ ] Medium [X] Low [ ] |
| **Consequences if not accepted** | The system will not be able to support automated tool selection. |
| **Dependencies** | None |

Table 2: CR_1: Feature Extraction Integration

#### 4.1.1 Methodology

The feature extraction process is implemented by invoking an external tool capable of analyzing smart contracts and producing structured feature vectors. These features are then formatted and prepared as input for the machine learning models.

#### 4.1.2 Expected Results

- automatic extraction of relevant contract features;
- standardized feature representation for further processing;
- minimal impact on the existing SmartBugs pipeline.

### 4.2 CR_2: Machine Learning-Based Tool Selection

| | |
|---|---|
| **Description** | Introduction of machine learning models to predict the most appropriate analysis tools based on extracted features. |
| **Motivation** | Currently, tool selection is manual and requires domain expertise. Machine learning models enable automated and data-driven selection of tools. |
| **Priority** | High [X] Medium [ ] Low [ ] |
| **Effort** | High [ ] Medium [X] Low [ ] |
| **Consequences if not accepted** | Tool selection remains manual, reducing usability and potentially leading to suboptimal analysis results. |
| **Dependencies** | CR_1 |

Table 3: CR_2: Machine Learning-Based Tool Selection

#### 4.2.1 Methodology

A set of pre-trained machine learning models is loaded from serialized files. Each model processes the extracted features and predicts the most suitable analysis tool. The predictions are then aggregated and filtered to remove invalid or duplicate entries.

#### 4.2.2 Expected Results

- automatic prediction of relevant tools;
- improved accuracy in tool selection;
- reduction of manual configuration effort.

### 4.3 CR_3: Integration into SmartBugs Workflow

| | |
|---|---|
| **Description** | Integration of the feature extraction and machine learning modules into the SmartBugs execution pipeline through a new intelligent command. |
| **Motivation** | The outputs of feature extraction and model inference must be connected to the existing SmartBugs workflow to enable automatic execution of selected tools. |
| **Priority** | High [X] Medium [ ] Low [ ] |
| **Effort** | High [X] Medium [ ] Low [ ] |
| **Consequences if not accepted** | The introduced components remain isolated and cannot be used within the SmartBugs pipeline. |
| **Dependencies** | CR_1, CR_2 |

Table 4: CR_3: Integration into SmartBugs Workflow

#### 4.3.1 Methodology

The SmartBugs orchestrator is extended to support a new execution mode. When enabled, the system extracts features, runs the machine learning models, selects the appropriate tools, and executes them using the existing analysis pipeline.

The integration is designed to be modular and non-intrusive, ensuring that the original workflow remains unchanged when the intelligent mode is not used.

#### 4.3.2 Expected Results

- seamless integration of intelligent tool selection into SmartBugs;
- preservation of backward compatibility;
- improved usability and automation of the analysis process.

# 5 Impact Analysis

To assess the effects of the proposed changes on the SmartBugs system, an impact analysis was conducted for each change request.

The analysis was performed through the following steps:

1. **Change request analysis**: each change request was examined to identify the responsibilities, behaviors, and configuration mechanisms expected to change.
2. **Reverse engineering**: the existing SmartBugs architecture was reconstructed from the source code in order to obtain the structural artifacts used in the analysis, namely package diagrams, class diagrams, and the dependency matrix among the main modules.
3. **Concept-to-module mapping**: the concepts introduced by each change request were mapped onto the existing modules by using the reverse-engineered artifacts.
4. **Dependency analysis**: the dependency matrix was used to identify direct dependency relations and possible impact propagation paths from the SIS to additional candidate modules.
5. **Source code analysis**: the source code of the involved and candidate modules was manually inspected to understand their responsibilities and to verify whether a structural dependency actually suggested a plausible need for modification.

For each change request, the following impact sets are identified:

- **Starting Impact Set (SIS)**: the set of modules initially assumed to be affected by the change request;
- **Candidate Impact Set (CIS)**: the set of modules estimated to be potentially impacted according to the adopted dependency-based impact analysis approach;
- **Actual Impact Set (AIS)**: the set of modules actually modified during implementation.

In this work, the **SIS** is identified by analyzing the content of the change request and mapping its concepts onto the existing architecture through reverse-engineered artifacts. Therefore, the SIS contains the modules that directly realize the behavior or configuration concerns addressed by the change request.

The **CIS** is then derived from the SIS through dependency analysis. More specifically, the reverse-engineered diagrams and the dependency matrix are used to identify modules that may be affected through direct dependency relations. These candidate modules are then validated by manually analyzing their source code, in order to reduce over-estimation and exclude modules whose dependencies are only structural and do not suggest any concrete need for modification.

## 5.1 CR_1: Integration of Feature Extraction Module

The first change request introduces a feature extraction component as a new module. At this stage, the component is developed independently from the existing SmartBugs orchestration workflow.

**Starting Impact Set (SIS)**

The change request analysis shows that the new functionality is encapsulated in a dedicated module and does not directly alter existing SmartBugs responsibilities. Reverse-engineered structural artifacts confirm that the feature extraction logic is introduced as a separate concern.

Therefore, the SIS for this change request contains only the newly introduced module:

- **sb.features**: feature extraction module.

$$SIS = \{\text{sb.features}\}$$

**Candidate Impact Set (CIS)**

Dependency analysis does not reveal any existing SmartBugs module that must be adapted to support this change request. Since the new module is introduced in isolation and does not require modifications to the current orchestration, configuration, or execution mechanisms, no additional candidates are identified.

Therefore, the CIS coincides with the SIS:

$$CIS = \{\text{sb.features}\}$$

**Actual Impact Set (AIS)**

The AIS will be determined after implementation. At design time, the expected actual impact is limited to the new module itself.

| Module | SIS | CIS | AIS |
|---|---|---|---|
| sb.features (new) | X | X | TBD |

Table 5: Impact sets for CR_1

**Discussion**

CR_1 has a low impact because it introduces a new module without affecting the existing SmartBugs workflow or its core modules.

## 5.2 CR_2: Machine Learning-Based Tool Selection

The second change request introduces the machine learning inference component responsible for predicting the most suitable analysis tools from the extracted features.

**Starting Impact Set (SIS)**

The change request analysis shows that the new responsibility is again encapsulated in a dedicated module. Reverse-engineered structural artifacts confirm that this functionality is introduced as a separate component and does not yet alter the existing SmartBugs orchestration.

Therefore, the SIS contains only the new inference module:

- **sb.models**: model inference module.

$$SIS = \{\text{sb.models}\}$$

**Candidate Impact Set (CIS)**

Although the model inference component conceptually depends on the output of the feature extraction module, this change request still introduces the component as a separate module and does not yet integrate it into the existing SmartBugs execution pipeline. Dependency analysis and manual code inspection do not indicate any required modification to existing SmartBugs modules.

Therefore, also in this case, the CIS coincides with the SIS:

$$CIS = \{\text{sb.models}\}$$

**Actual Impact Set (AIS)**

The AIS will be determined after implementation. At design time, the expected actual impact is limited to the new inference module.

| Module | SIS | CIS | AIS |
|---|---|---|---|
| sb.models (new) | X | X | TBD |

Table 6: Impact sets for CR_2

**Discussion**

CR_2 also has limited impact because it introduces a new module without modifying the current orchestration, configuration, or execution mechanisms of SmartBugs.

## 5.3 CR_3: Integration into SmartBugs Workflow

The third change request integrates the new feature extraction and model inference components into the SmartBugs execution workflow. Unlike CR_1 and CR_2, this change affects existing orchestration and configuration responsibilities.

**Starting Impact Set (SIS)**

The SIS was identified by analyzing the content of the change request and locating the modules that directly realize the responsibilities affected by the proposed intelligent workflow. This identification was supported by reverse-engineered package and class diagrams, together with manual analysis of the source code responsibilities implemented by the involved modules.

The analysis shows that the change directly affects:

- **sb.smartbugs**, since the orchestration logic must be extended to execute feature extraction, model inference, and dynamic tool selection;
- **sb.cli**, since the command-line interface must accept and forward the new configuration parameters required by the intelligent workflow;
- **sb.settings**, since the configuration module must be extended to store and propagate the new runtime settings.

Therefore:

$$SIS = \{\text{sb.smartbugs, sb.cli, sb.settings}\}$$

**Candidate Impact Set (CIS)**

The CIS was derived from the SIS through dependency analysis based on the reverse-engineered diagrams and the dependency matrix. The resulting candidate modules were then examined through manual source code analysis in order to determine whether their dependency relations implied a realistic need for modification.

This process did not identify additional modules beyond the SIS itself.

In particular, `sb.tasks` was considered during the analysis because it depends structurally on `sb.settings`. However, manual analysis of the source code showed that `Task` only stores a reference to the global settings object and does not implement orchestration logic, intelligent tool selection, or configuration management behavior that would need to be adapted for this change request. Therefore, `sb.tasks` was excluded from the CIS.

Similarly, modules such as `sb.tools`, `sb.analysis`, and `sb.docker` participate in the execution pipeline, but the reverse-engineered dependencies and source code analysis do not suggest that they must be modified as a direct consequence of the proposed integration. Including them in the CIS would therefore result in over-estimation.

As a result, no additional plausible candidates emerged beyond the SIS:

$$CIS = \{\text{sb.smartbugs, sb.cli, sb.settings}\}$$

**Actual Impact Set (AIS)**

The AIS will be established after implementation, once the modules effectively modified by CR_3 are known.

| Module | SIS | CIS | AIS |
|---|---|---|---|
| sb.cli | X | X | TBD |
| sb.smartbugs | X | X | TBD |
| sb.settings | X | X | TBD |

Table 7: Impact sets for CR_3

**Discussion**

CR_3 is the most critical change request because it integrates the new intelligent workflow into the existing SmartBugs architecture. However, the estimated impact remains limited and controlled. The analysis highlights only a small set of core modules as plausibly affected, which helps contain implementation effort and supports focused regression testing.

## 5.4 Overall Impact

The impact analysis shows that:

- CR_1 and CR_2 introduce new components without impacting existing SmartBugs modules;
- CR_3 affects only a limited number of core modules directly responsible for orchestration and configuration;
- the majority of the SmartBugs system remains unchanged.

Overall, the proposed solution appears modular and non-intrusive: it extends the functionality of the original system while preserving the stability of most of its existing structure.

# 6 Testing Strategy and Assessment

Testing activities are documented in dedicated testing documents, as required by the project guidelines. In particular, the testing documentation includes:

- a test plan describing the testing strategy, adequacy criteria, regression testing approach, and category partition;
- a pre-modification test case specification;
- a pre-modification test execution report;
- a post-modification test case specification;
- execution reports for the modified parts and for regression testing.

At this stage, the existing SmartBugs test suite has been assessed to define the pre-modification testing baseline. The project already includes a pytest-based test suite, with tests covering key parts of the orchestration logic, including file collection, task generation, tool loading, and main workflow execution.

The existing tests are mainly unit and component tests. Although they simulate several interactions between modules, they rely on mocking for external dependencies such as Docker execution and Solidity compiler handling. Therefore, additional tests will be introduced to validate the new intelligent workflow involving feature extraction, machine learning inference, tool aggregation, and integration into the SmartBugs pipeline.

Detailed test cases, execution results, and regression testing results are provided in the separate testing documents.

# 7 Implementation of the Changes

After analysing the possible impact of the proposed change requests, the implementation was carried out on a forked version of the SmartBugs repository.

The implementation followed the modular structure defined during the design phase. The goal was to introduce the intelligent tool-selection workflow while preserving the original SmartBugs execution pipeline. For this reason, the implementation was divided into three main steps, corresponding to the three change requests previously defined:

- integration of a feature extraction wrapper;
- integration of machine learning-based tool selection;
- integration of the new intelligent workflow into the SmartBugs command-line execution.

The implementation was mainly performed by adding new modules and by extending the orchestration and configuration logic only where necessary.

## 7.1 CR_1: Feature Extraction Module

The first change request introduced a new module, `sb.features`, responsible for interacting with SCsVulLyzer.

SCsVulLyzer is an external feature extraction tool that analyzes Solidity smart contracts and produces a set of structural features. The new wrapper is responsible for invoking SCsVulLyzer through a subprocess, capturing its output, and converting the extracted features into a Python dictionary that can be used by the rest of the system.

The feature extraction module performs the following operations:

- validates the existence of the input Solidity contract;
- validates the availability of the SCsVulLyzer entry point;
- invokes SCsVulLyzer using the current Python interpreter;
- parses the output produced by SCsVulLyzer;
- raises a dedicated exception in case of missing files, command failures, empty output, or invalid output format.

During implementation, it was observed that SCsVulLyzer did not return a Python dictionary directly, but printed its features as key-value pairs in the standard output. Therefore, the parsing logic was adapted to support this format. The final parser converts lines such as:

```
Opcode Count Features_ADD: 65
Number of total_lines: 131
```

into a structured dictionary.

### 7.1.1 Results

The implementation of CR_1 was successful. The new module isolates the dependency on SCsVulLyzer and provides a reusable interface for the following change requests.

The updated impact sets are shown in Table 8.

| Module | Candidate Impact Set | Discovered Impact Set | Actual Impact Set |
|---|---|---|---|
| sb.features (new) | X | | X |

Table 8: Actual impact set for CR_1 after implementation

The following impact analysis metrics were computed:

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{1}{1} = 100\%$$

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{1}{1} = 100\%$$

## 7.2 CR_2: Machine Learning-Based Tool Selection

The second change request introduced a new module, `sb.models`, responsible for selecting the most suitable SmartBugs tools using pre-trained machine learning models.

The provided models are serialized as `.joblib` files and are implemented as `scikit-learn` pipelines. During the implementation, the models were inspected through their `feature_names_in_` attribute. This made it possible to identify the exact list of 41 features required by the models.

The new `sb.models` module provides the following functionalities:

- selection of the 41 required features from the SCsVulLyzer output;
- loading of serialized `.joblib` models;
- execution of model predictions;
- aggregation of predicted tools;
- removal of duplicate predictions;
- removal of non-informative predictions such as `nessun_tool`;
- optional filtering of unsupported tools.

The dependency `scikit-learn` was pinned to version 1.6.1 because the provided models were serialized with that version. Loading the models with a newer version caused compatibility issues during deserialization.

### 7.2.1 Results

The implementation of CR_2 was successful. The new module can load the provided machine learning models, select the required feature subset, execute predictions, and produce a clean list of selected SmartBugs tools.

The updated impact sets are shown in Table 9.

| Module | Candidate Impact Set | Discovered Impact Set | Actual Impact Set |
|---|---|---|---|
| sb.models (new) | X | | X |

Table 9: Actual impact set for CR_2 after implementation

The following impact analysis metrics were computed:

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{1}{1} = 100\%$$

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{1}{1} = 100\%$$

The module `sb.models` consumes the feature dictionary produced by `sb.features`, but no modification to the feature extraction module was required during this change request.

## 7.3 CR_3: Integration into SmartBugs Workflow

The third change request integrated the feature extraction and model-based tool selection modules into the SmartBugs execution workflow.

The integration required modifications to the command-line interface, the configuration object, and the main orchestration logic. In particular:

- `sb.cli` was extended with new command-line options;
- `sb.settings` was extended with new configuration parameters;
- `sb.smartbugs` was extended with the intelligent tool-selection workflow;
- `sb.features` and `sb.models`, implemented in the previous change requests, were invoked from the orchestrator.

A new command-line flag, `--smart-select`, was introduced. When this flag is enabled, SmartBugs first collects the input files and then executes the intelligent tool-selection phase before loading tools and generating tasks.

The implemented workflow is the following:

1. collect the input files using the existing SmartBugs file collection mechanism;
2. for each Solidity file, extract features using SCsVulLyzer;
3. select the subset of features required by the machine learning models;
4. execute the pre-trained models and collect their predictions;
5. aggregate and filter the predicted tools;
6. normalize tool identifiers when necessary;
7. update the list of tools in the SmartBugs settings;
8. continue with the original SmartBugs task generation and execution pipeline.

The new workflow can be executed with the following command:

```
python -m sb --smart-select -f samples/0.8.24/ERC20.sol --continue-on-errors
```

During manual validation, the intelligent workflow successfully extracted the contract features, executed the model-based tool-selection phase, selected `smartcheck`, and generated one SmartBugs task. The execution then reached the original Docker-based analysis phase.

### 7.3.1 Results

The implementation of CR_3 was successful. The new intelligent execution mode is available through the command line and remains backward compatible with the original manual workflow. When `--smart-select` is not enabled, SmartBugs continues to behave as in the original version.

The updated impact sets are shown in Table 10.

| Module | Candidate Impact Set | Discovered Impact Set | Actual Impact Set |
|---|---|---|---|
| sb.cli | X | | X |
| sb.settings | X | | X |
| sb.smartbugs | X | | X |

Table 10: Actual impact set for CR_3 after implementation

The following impact analysis metrics were computed:

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{3}{3} = 100\%$$

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{3}{3} = 100\%$$

### 7.4 Summary of Implementation Results

The implementation confirmed the initial design assumption that the intelligent workflow could be integrated mainly at the orchestration level.

The main implemented changes are:

- a new feature extraction wrapper in `sb.features`;
- a new machine learning tool-selection module in `sb.models`;
- new configuration parameters in `sb.settings`;
- new command-line options in `sb.cli`;
- new orchestration logic in `sb.smartbugs`.

The original SmartBugs execution pipeline was preserved. The existing modules responsible for tool loading, task generation, Docker execution, and analysis execution were not directly modified.

This confirms that the proposed solution is modular and that the impact of the change is mostly limited to the newly introduced components and to the orchestration layer.

## 8 Conclusions

This project evolved SmartBugs by introducing an intelligent tool-selection mechanism based on smart contract feature extraction and machine learning models.

The implemented extension allows SmartBugs to analyze the input contract, extract structural features through SCsVulLyzer, execute a set of pre-trained models, and automatically select the tools to be executed. The selected tools are then passed to the original SmartBugs workflow, preserving the existing task generation and Docker-based execution pipeline.

The implementation was organized according to three change requests. The first change request introduced a wrapper for SCsVulLyzer, the second introduced the machine learning-based tool-selection module, and the third integrated the new workflow into the SmartBugs command-line execution.

The impact analysis was useful to identify the modules potentially affected by the evolution. After implementation, the Actual Impact Set confirmed that the changes were localized in the new modules and in the orchestration/configuration layer, while the rest of the SmartBugs execution pipeline could be preserved without modification.

Testing activities are documented separately, as required by the project guidelines. Unit tests were added for the new feature extraction and model selection modules, while existing SmartBugs tests were reused to support regression testing of the original workflow.

The final result is a modular and backward-compatible extension of SmartBugs. The main limitation of the current implementation is the dependency on external components, namely SCsVulLyzer, the pre-trained `.joblib` models, `scikit-learn` version 1.6.1, and Docker for the final execution of the selected analysis tools. Future improvements may include a more robust management of external dependencies, additional end-to-end integration tests involving the external execution environment, and alternative strategies for aggregating model predictions.